Шинжлэх Ухаан Технологийн Их Сургууль
Мэдээлэл, Холбооны Технологийн Сургууль



F.CSM302 Үйлдлийн систем

Бие даалт №1

Designing a Virtual Memory Manager

Хуваарилалтын код: II-4-15

Шалгасан багш: Г.Ганбат /F.CS10/

Гүйцэтгэсэн оюутан: Э.Мөнх-Оргил /B242270130/
Д.Ундармаа /B242270044/
В.Хүсэлбилэг /B241910007/
С.Сүхбат /B241940044/
М.Хатансайхан /B231940049/

Улаанбаатар хот
2026 он

Агуулга

Хураангуй **2**

1 Удиртгал **2**
1.1 Зорилго . . . 2
1.2 Ач холбогдол . . . 2
1.3 Хамрах хүрээ . . . 2

2 Сурах бичгийн шаардлага **3**

3 Онолын үндэс **3**
3.1 Виртуал санах ой ба paging . . . 3
3.2 Өгөгдлийн бүтэц . . . 4
3.3 Хаяг хөрвүүлэх алгоритм . . . 4
3.4 Page replacement: FIFO ба LRU . . . 5
3.5 Гараар бодсон сурах бичгийн жишээ . . . 5

4 Системийн зохиомж **6**

5 Хэрэгжилт **7**
5.1 Хэл ба орчин . . . 7
5.2 Гол шийдлүүд . . . 7
5.3 Жишээ код: VMM.access() . . . 7

6 Туршилт ба UI дүрслэл **8**
6.1 Юуг өөрчилж турших вэ . . . 8
6.2 Юуг ажиглах вэ . . . 8
6.3 Туршилтууд . . . 9

7 Туршилтын арга **10**

8 Үр дүн **10**
8.1 Автомат тестийн үр дүн . . . 10
8.2 Туршилтын үр дүн . . . 11
8.3 Screenshot (Xvfb дээр бодитоор ажиллуулж авсан) . . . 14

9 Хэлэлцүүлэг **15**
9.1 Үр дүнгийн тайлбар . . . 15
9.2 Хязгаарлалт ба мэдэгдэж буй асуудлууд . . . 15

10 Дүгнэлт **15**

Ашигласан материал **16**

Хавсралт А. Ажлын төлөвлөгөө **16**

Хураангуй

Асуудал. Сурах бичгийн 10-р бүлгийн Programming Project нь виртуал санах ойн менежер бичихийг шаарддаг. Програм нь 32 битийн логик хаягийн баруун 16 битийг 8 битийн page дугаар, 8 битийн offset болгон задалж, TLB болон page table ашиглан физик хаяг руу хөрвүүлнэ. Page fault гарвал backing store-ооc page ачаалж, санах ой дүүрсэн үед page солино.

Шийдэл. Python 3 хэл дээр физик санах ойг бүрэн загварчилсан `vmm.py` цөм модуль хөгжүүлэв. Түүнийг ашигладаг номын гаралттай ижил форматтай консол програм (`cli.py`), FIFO/LRU-г харьцуулах туршилтын скрипт (`experiment.py`), tkinter-ийн **page/frame animation** бүхий interactive UI (`gui.py`) бичсэн. Хэрэглэгч хаяг эсвэл reference string, frame-ийн тоо (1–256), TLB-ийн хэмжээ, replacement бодлого, backing store файлыг сольж дахин туршиж болно.

Туршилт. Сурах бичгийн албан ёсны `addresses.txt`, `BACKING_STORE.bin`, `correct.txt` файлууд дээр туршив. Үүнээс гадна номын хоёр reference string, locality-тэй 1000 хаяг, захын болон алдаатай оролт, 24 автомат тест ашиглав.

Гол үр дүн. Номын 1000 хаягийг 256 frame-тэй хөрвүүлэхэд гаралт `correct.txt`-тэй **1000/1000 мөр яг таарав** (Page Faults = 244, TLB Hits = 54). 128 frame-тэй үед FIFO 538, LRU 539 fault гарсан бөгөөд 1000 утга бүгд зөв хэвээр байлаа. Номын reference string дээр 3 frame-тэй үед **FIFO 15, LRU 12** fault гарч номын хариутай таарав. Belady-ийн аномали ч давтагдсан. Locality-тэй өгөгдөлд 16 frame үед LRU 22.6%, FIFO 27.7% fault rate гарсан. Харин санамсаргүй хаягтай номын өгөгдөлд хоёр алгоритмын ялгаа бараг байсангүй.

Гол үр дүн

1000/1000	15 / 12	9 → 10	22.6%
Номын correct.txt-тэй таарсан мөр (256 frame)	Номын жишээ: FIFO / LRU fault (номтой таарсан)	Belady-ийн аномали (FIFO, 3 → 4 frame)	LRU fault rate (16 frame) FIFO-д 27.7%

Хязгаарлалт (товч). Ном шаардаагүй тул бичих үйлдэл (dirty bit) болон олон процесс хэрэгжээгүй. TLB нь page replacement-тэй ижил бодлогоор солигддог.

1. Удиртгал

1.1 Зорилго

Виртуал санах ойн хаяг хөрвүүлэлт (address translation), TLB, demand paging болон page replacement алгоритмыг өөрсдөө хэрэгжүүлнэ. Хэрэгжүүлсэн програмаа номын албан ёсны өгөгдлөөр баталгаажуулж, ажиллагааг нь алхам алхмаар дүрслэн харуулна. Мөн параметр өөрчлөхөд гүйцэтгэл хэрхэн өөрчлөгдөхийг туршилтаар нотолно.

1.2 Ач холбогдол

Орчин үеийн ихэнх үйлдлийн систем paging-д суурилсан виртуал санах ой ашигладаг. Лекц дээр онолоор үзсэн page table, TLB, page fault, FIFO/LRU, Belady-ийн аномали зэрэг ойлголтыг ажиллаж буй програм дээр харж, тоон үр дүнгээр баталгаажуулах нь ойлголтыг гүнзгийрүүлнэ.

1.3 Хамрах хүрээ

$2^{16} = 65\,536$ байт виртуал хаягийн орон зай, 256 байтын page, 256 page, 1–256 frame, дурын хэмжээтэй TLB, FIFO ба LRU бодлого. Бичих үйлдэл (dirty page), олон процесс, олон түвшний

page table хамрах хүрээнд ороогүй. Эдгээрийг ном ч шаардаагүй.

2. Сурах бичгийн шаардлага

Сурах бичгийн 10-р бүлгийн „Designing a Virtual Memory Manager“ төслийн үндсэн шаардлага ба тэдгээрийн хэрэгжилт:

№	Шаардлага	Хэрэгжилт	Төлөв
1	Баруун 16 битийг 8 битийн page, 8 битийн offset болгон задлах	`a = addr & 0xFFFF; page = a >> 8; offset = a & 0xFF`	Хэрэгжсэн
2	2^8 page table оролт, 2^8 байт page/frame, 256 frame (65 536 байт физик санах ой)	`PAGE_SIZE = NUM_PAGES = MAX_FRAMES = 256`, `memory = bytearray(frames*256)`	Хэрэгжсэн
3	16 оролттой TLB. TLB miss үед page table	`VMM(tlb_size=16)`. TLB → page table дараалал	Хэрэгжсэн
4	Page fault үед `BACKING_STORE.bin`-оос 256 байт уншиж, хоосон frame-д хадгалах (demand paging)	`memory[frame*256 ...] = store[page*256 ...]`	Хэрэгжсэн
5	Физик санах ой дахь signed byte утгыг гаргах	`b - 256 if b > 127 else b`	Хэрэгжсэн
6	Гаралт: логик хаяг, физик хаяг, утга. Page-fault rate, TLB hit rate	`cli.py` номын форматаар	Хэрэгжсэн
7	Гаралтыг `correct.txt`-ээр шалгах	`tools/check_correct.py`: 1000/1000	Хэрэгжсэн
8	Өөрчлөлт: 128 frame, page replacement (FIFO эсвэл LRU)	`--frames 1..256 --policy FIFO|LRU`	Хэрэгжсэн
9	TLB-ийн солих бодлого (FIFO эсвэл LRU)	TLB сонгосон бодлогоор солигдоно	Хэрэгжсэн

Багшийн хуваарилалтад заасан нэмэлт шаардлага: *„Address эсвэл reference sequence, frame-ийн тоо, replacement сонголтыг өөрчилж, page/frame-ийн өөрчлөлт, hit, fault-ийг алхамчилж харуулна.“* Энэ шаардлагыг GUI-ийн reference string горим, frame/TLB/policy тохиргоо, Алхам ба Анимейшн товчоор хангасан.

3. Онолын үндэс

3.1 Виртуал санах ой ба paging

Процесс бүр өөрийн логик хаягийн орон зайтай. Үйлдлийн систем түүнийг ижил хэмжээтэй *page*-д хувааж, физик санах ойн *frame*-д байрлуулна. Page table нь page дугаарыг frame дугаар руу буулгадаг. Санах ой руу хандах бүрт page table руу давхар хандахгүйн тулд сүүлийн хөрвүүлэлтүүдийг *TLB* хэмээх жижиг, хурдан кэшэд хадгална.

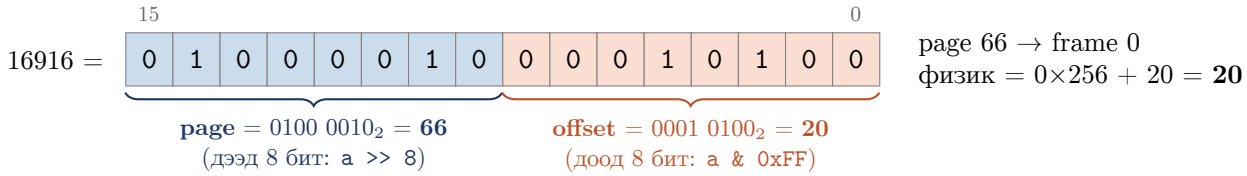
Нэр томьёо	English	Монгол
Page / Frame	Fixed-size block of virtual / physical memory	Виртуал / физик санах ойн тогтмол хэмжээтэй хэсэг (энд 256 байт)
Page table	Maps page number → frame number	Page бүр аль frame-д байгааг заасан хүснэгт
TLB	Small fast cache of recent page→frame translations	Сүүлийн хөрвүүлэлтүүдийг хадгалах хурдан кэш
Page fault	Trap when a referenced page is not in memory	Хандсан page санах ойд байхгүй үед үүсэх тасалдал
Demand paging	Load a page only when first referenced	Page-ийг анх хандах үед нь л ачаалах
Backing store	Secondary storage holding all pages	Бүх page хадгалагддаг хоёрдогч санах ой (энд `BACKING_STORE.bin`)
FIFO	Replace the page loaded earliest	Хамгийн эрт ачаалсан page-ийг солих
LRU	Replace the page unused for the longest time	Хамгийн удаан хэрэглээгүй page-ийг солих
Belady's anomaly	More frames can cause more faults (FIFO)	Frame нэмэхэд fault өсөх үзэгдэл
Locality	Programs reuse a small working set for a while	Програм тодорхой хугацаанд цөөн page-д давтан хандах шинж

3.2 Өгөгдлийн бүтэц

Нэр	Хэмжээс	Утга
page_table	256	Page бүрийн frame дугаар, санах ойд байхгүй бол -1
memory	$F \times 256$	Физик санах ой (`bytearray`). Frame f нь $[f \cdot 256,\ (f+1) \cdot 256)$ байт
frames	$F \times 3$	Frame бүрийн `[page, loaded, used]`. `loaded`: ачаалсан цаг, `used`: сүүлд хандсан цаг
tlb	$\leq T$	`[page, frame, loaded, used]` оролтууд
clock	1	Хандалт бүрт 1-ээр өсөх логик цаг
store	65 536	Backing store (`BACKING_STORE.bin` эсвэл жишиг өгөгдөл)

3.3 Хаяг хөрвүүлэх алгоритм

Жишээ болгон номын эхний хаяг 16916-г авч үзье. Баруун 16 битийг хоёр 8 битэд хуваана:

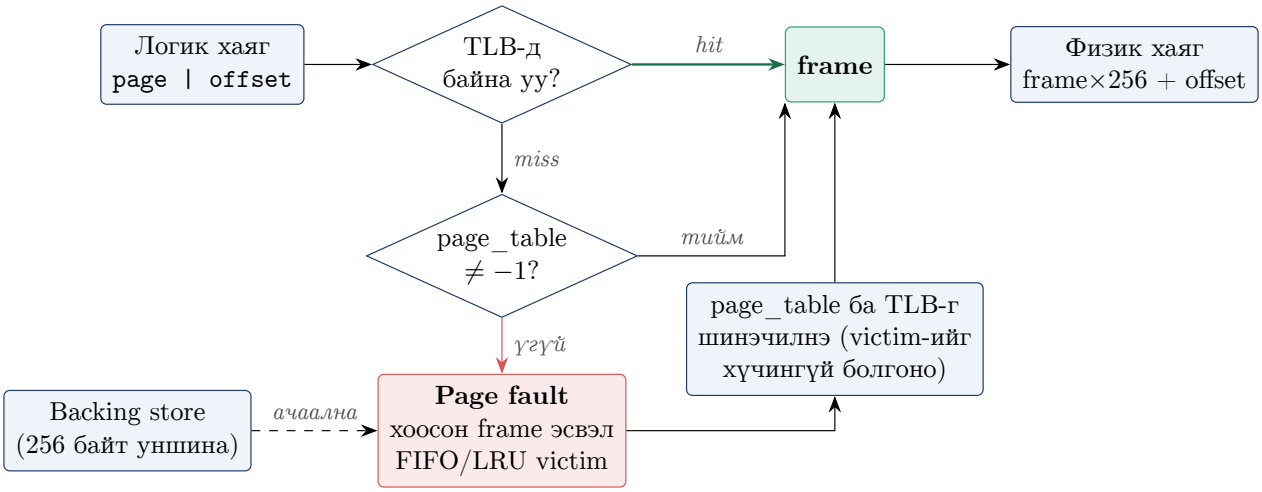


Зураг 1. 16 битийн логик хаягийг page ба offset болгон задлах (16916 → page 66, offset 20)

Ерөнхий алгоритм:

1. $a = \text{addr} \,\&\, \text{0xFFFF}$, $\text{page} = a \gg 8$, $\text{offset} = a \,\&\, \text{0xFF}$
2. TLB-ээс page-ийг хайна. Олдвол frame-ийг авна → **TLB hit**

3. Олдохгүй бол page_table[page]-ийг шалгана. ≥ 0 бол frame-ийг авна
4. -1 бол **PAGE FAULT**: хоосон frame байвал түүнийг авна, үгүй бол FIFO/LRU-гаар victim сонгож, түүний page table ба TLB оролтыг хүчингүй болгоно. Дараа нь backing store-оос 256 байтыг тэр frame руу хуулна
5. TLB-д (page, frame)-ийг нэмнэ. TLB дүүрсэн бол FIFO/LRU-гаар нэг оролтыг гаргана
6. physical $=$ frame $\times$ 256 $+$ offset, value $=$ signed(memory[physical])



Зураг 2. Хаяг хөрвүүлэх урсгал: TLB → page table → page fault

Хандалт бүрийн хугацааны нарийвчлал нь TLB хайлтад $O(T)$, victim сонголтод $O(F)$ байна.

3.4 Page replacement: FIFO ба LRU

FIFO нь `loaded`-ийн хамгийн бага утгатай frame-ийг, LRU нь `used`-ийн хамгийн бага утгатай frame-ийг victim болгоно. LRU нь *stack алгоритм* тул n frame-д байгаа page-ийн олонлог нь $n+1$ frame-ийнхийн дэд олонлог байдаг. Иймээс frame нэмэхэд fault хэзээ ч өсдөггүй. FIFO-д энэ шинж байхгүй тул Belady-ийн аномали гарч болно.

3.5 Гараар бодсон сурах бичгийн жишээ

Reference string `7 0 1 2 0 3 0 4 2 3 0 3 2 1 2 0 1 7 0 1`, 3 frame. Багана нь F0, F1, F2 frame-ийн агуулга. **F** нь page fault, $(-p)$ нь гаргасан page:

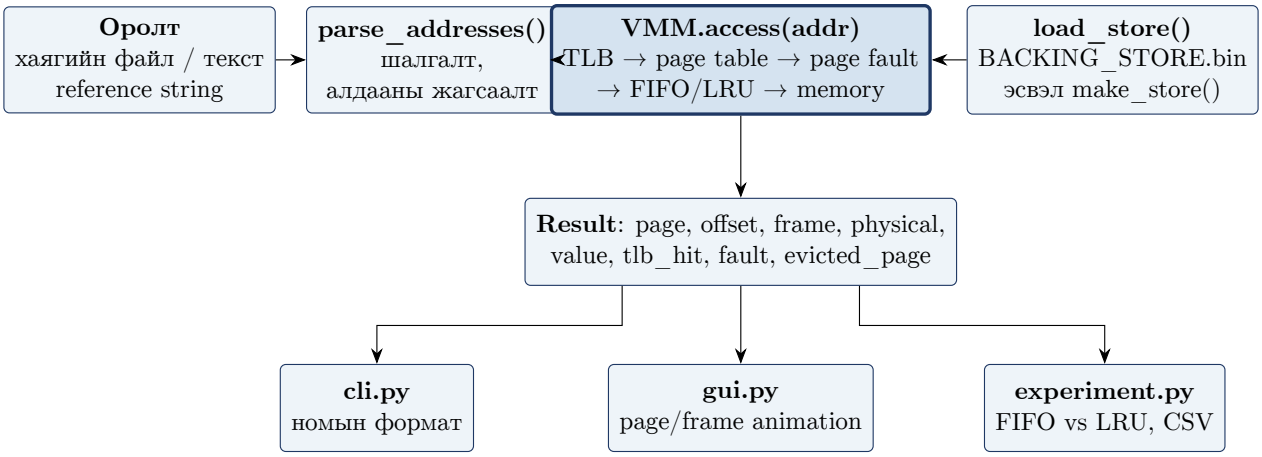
#	Page	FIFO: F0 F1 F2	Үр дүн	LRU: F0 F1 F2	Үр дүн
1	7	7 – –	**F**	7 – –	**F**
2	0	7 0 –	**F**	7 0 –	**F**
3	1	7 0 1	**F**	7 0 1	**F**
4	2	2 0 1	**F** (-7)	2 0 1	**F** (-7)
5	0	2 0 1	hit	2 0 1	hit
6	3	2 3 1	**F** (-0)	2 0 3	**F** (-1)
7	0	2 3 0	**F** (-1)	2 0 3	hit
8	4	4 3 0	**F** (-2)	4 0 3	**F** (-2)
9	2	4 2 0	**F** (-3)	4 0 2	**F** (-3)
10	3	4 2 3	**F** (-0)	4 3 2	**F** (-0)
11	0	0 2 3	**F** (-4)	0 3 2	**F** (-4)
12	3	0 2 3	hit	0 3 2	hit
13	2	0 2 3	hit	0 3 2	hit

#	Page	FIFO: F0 F1 F2	Үр дүн	LRU: F0 F1 F2	Үр дүн
14	1	0 1 3	**F** (−2)	1 3 2	**F** (−0)
15	2	0 1 2	**F** (−3)	1 3 2	hit
16	0	0 1 2	hit	1 0 2	**F** (−3)
17	1	0 1 2	hit	1 0 2	hit
18	7	7 1 2	**F** (−0)	1 0 7	**F** (−2)
19	0	7 0 2	**F** (−1)	1 0 7	hit
20	1	7 0 1	**F** (−2)	1 0 7	hit
Нийт			**15 fault**		**12 fault**

Гараар бодсон дүн номын хариутай (FIFO 15, LRU 12) таарсан бөгөөд програмын гаралт ч мөн адил гарсан (8-р бүлэг).

4. Системийн зохиомж

Систем нь UI-аас хамааралгүй **цөм** (`vmm.py`) ба түүнийг ашигладаг гурван **интерфейс**-ээс (CLI, GUI, туршилт) бүрдэнэ. GUI, CLI хоёулаа ижил `VMM` классыг ашигладаг тул дүрслэл нь бодит ажиллагаатай шууд холбогдсон.



Зураг 3. Компонентууд ба өгөгдлийн урсгал

Файл	Үүрэг	Мөр
`src/vmm.py`	Цөм: `VMM` класс (`access()`, физик санах ой, статистик), `parse_addresses()`, `load_store()`, `VMMError`	128
`src/cli.py`	Консол програм: `--frames --policy --tlb --store --refs --summary`	58
`src/gui.py`	tkinter UI: тохиргоо, Алхам / Анимейшн / Reset, frame grid, TLB, хаягийн задлал, хүснэгт	162
`src/experiment.py`	Frame × {FIFO, LRU} туршилт, хүснэгт ба CSV	60
`tests/test_vmm.py`	24 автомат тест	182
`tools/check_correct.py`	Гаралтыг номын `correct.txt`-тэй мөр бүрээр харьцуулна	53
`tools/gen_examples.py`	Өөрсдийн туршилтын өгөгдлийг seed=2026-аар үүсгэнэ	44
`examples/book/`	Сурах бичгийн албан ёсны `addresses.txt`, `BACKING_STORE.bin`, `correct.txt`	—

5. Хэрэгжилт

5.1 Хэл ба орчин

Python 3.10+ (стандарт сан: `dataclasses`, `re`, `argparse`, `tkinter`, `unittest`). Гадны dependency байхгүй, build алхам шаардлагагүй. Тайлангийн графикуудыг `docs/results/*.csv` үр дүнгээс LaTeX (pgfplots)-оор зурсан.

5.2 Гол шийдлүүд

- **Физик санах ой ба demand paging.** `memory = bytearray(frames × 256)`. Page fault гарахад backing store-оос 256 байтыг frame руу хуулж, утгыг зөвхөн физик санах ойгоос уншдаг. Иймд frame-ийн буруу зураглал гарвал утга нь шууд буруу гарна.
- **Логик цаг.** `clock` хандалт бүрт өснө. Frame болон TLB-ийн оролт бүр `loaded`, `used` цагтай. FIFO нь `loaded`-ийн, LRU нь `used`-ийн минимумаар victim сонгодог тул ганц `_victim()` функц хоёр бодлогыг хоёуланг нь хэрэгжүүлнэ.
- **Frame олголт.** Хоосон frame-ийг 0-ээс эхлэн дараалан олгодог. Иймд 256 frame-тэй үед физик хаяг номын `correct.txt`-тэй яг таарна.
- **TLB-ийн тууштай байдал.** Page гаргахад (evict) түүний TLB оролтыг устгадаг. Ингэснээр TLB хэзээ ч санах ойд байхгүй page руу заахгүй (`test_eviction_clears_tlb`).
- **Reference string горим.** `parse_addresses(text, refs=True)` нь тоо бүрийг page дугаар (0–255) гэж үзэж page×256 хаяг болгоно. Номын FIFO/LRU жишээг шууд шалгах боломжтой.
- **Алдааны боловсруулалт.** Хэрэглэгчид харуулах бүх алдаа `VMMError(ValueError)` төрөлтэй. Оролтын алдааг дугаар, утга, шалтгааны хамт жагсаана. Алдаа гарвал CLI нь exit code 2-оор гарч, GUI нь мэдэгдлийн цонх гаргана.
- **GUI.** 256 хүртэлх frame-ийг 16 баганатай grid-д зурна (scrollbar-тай, идэвхтэй frame руу автоматаар гүйлгэнэ). Өнгө: ногоон = hit, улаан = fault, шар хүрээ = солилт. `examples/book/BACKING_STORE.bin` байвал эхлэхдээ автоматаар ачаална.

5.3 Жишээ код: VMM.access()

```
def access(self, address):
    self.clock += 1; t = self.clock
    a = address & 0xFFFF                                # баруун 16 бит
    page, offset = a >> 8, a & 0xFF
    i = next((k for k, e in enumerate(self.tlb) if e[0] == page), -1)
    if i >= 0:                                          # TLB hit
        tlb_hit, frame = True, self.tlb[i][1]; self.tlb[i][3] = t
    else:
        frame = self.page_table[page]
        if frame < 0:                                   # page fault
            fault = True
            frame = next((k for k, f in enumerate(self.frames) if f[0] < 0), -1)
            if frame < 0:                               # санах ой дүүрсэн -> солилт
                frame = self._victim(self.frames, 1, 2)
                evicted = self.frames[frame][0]
                self.page_table[evicted] = -1
                self.tlb = [e for e in self.tlb if e[0] != evicted]
            src, dst = page * PAGE_SIZE, frame * PAGE_SIZE   # demand paging
            self.memory[dst:dst + PAGE_SIZE] = self.store[src:src + PAGE_SIZE]
            self.frames[frame] = [page, t, t]; self.page_table[page] = frame
        if len(self.tlb) >= self.tlb_size:
```

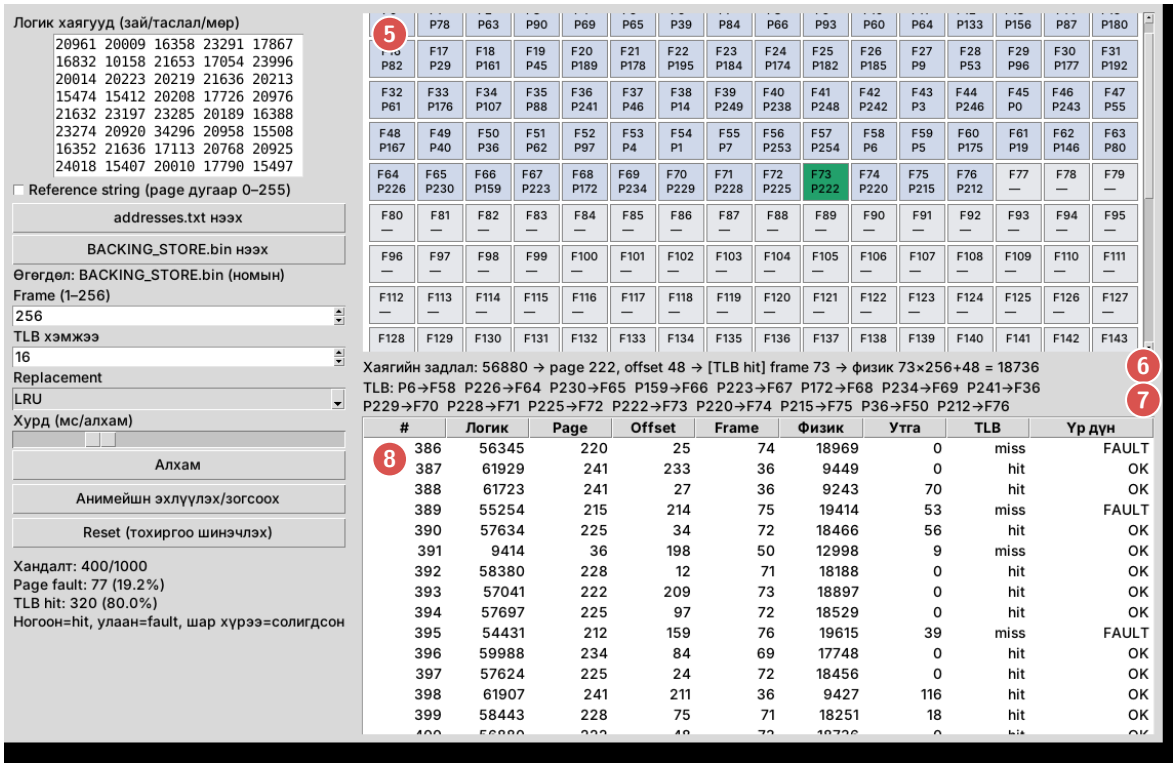
```
            self.tlb.pop(self._victim(self.tlb, 2, 3))
        self.tlb.append([page, frame, t, t])
    self.frames[frame][2] = t
    physical = frame * PAGE_SIZE + offset
    b = self.memory[physical]                          # физик санах ойгоос уншина
    ...  # статистик, Result(..., b - 256 if b > 127 else b, ...)
```

6. Туршилт ба UI дүрслэл

6.1 Юуг өөрчилж турших вэ

- **Оролт:** хаягийн жагсаалт (гараар эсвэл файлаас), **reference string** горим (page дугаар 0–255)
- **Frame**: 1–256, **TLB хэмжээ**: 1–64, **Replacement**: FIFO / LRU
- **BACKING_STORE.bin** файл, анимейшний **хурд** (50–1500 мс)
- **Алхам** (нэг хаяг), **Анимейшн** (эхлүүлэх/зогсоох), **Reset** (тохиргоог хэрэгжүүлж эхнээс нь эхлүүлэх)

6.2 Юуг ажиглах вэ



Зураг 4. Програмын цонхны хэсгүүд (256 frame, LRU, locality өгөгдөл, 400 алхмын дараа)

№	Хэсэг	Юуг харуулах
1	Логик хаягууд	Хаяг эсвэл reference string оруулах талбар, файлаас ачаалах товч
2	Тохиргоо	Frame (1–256), TLB хэмжээ, FIFO/LRU, хурд
3	Удирдлага	Алхам, Анимейшн эхлүүлэх/зогсоох, Reset
4	Статистик	Хандалт, page fault ба TLB hit-ийн тоо, хувь
5	Frame grid	`F#`/`P#`. **Ногоон** = hit, **улаан** = fault, **шар хүрээ** = солилт, саарал = хоосон
6	Хаягийн задлал	`56880 → page 222, offset 48 → [TLB hit] frame 73 → физик 18736`
7	TLB	Одоогийн `P→F` оролтууд
8	Хандалтын хүснэгт	#, логик, page, offset, frame, физик, утга, TLB hit/miss, `FAULT (-P7)`

6.3 Туршилтууд

№	Туршилт	Тохиргоо	Юуг харуулах
1	Номын албан ёсны өгөгдөл (`examples/book/`)	256, 128 fr	`correct.txt`-тэй мөр бүрээр таарах
2	Номын „How to begin“ 7 тоо	—	page/offset зөв задрах
3	Номын reference string (`ref_textbook.txt`)	3 fr, FIFO/LRU	FIFO 15, LRU 12 fault
4	Belady (`ref_belady.txt`)	1–6 fr, FIFO/LRU	FIFO: 3 → 4 frame үед 9 → 10
5	Захын утга (`boundary.txt`)	128 fr, FIFO	0, 65535, 65536, $2^{32}-1$ хаягууд
6	Алдаатай оролт (`bad_input.txt`, `empty.txt`)	—	Ойлгомжтой мессеж, exit 2
7	Frame-ийн нөлөө: номын хаяг ба locality өгөгдөл	8–256 fr	LRU vs FIFO, locality-ийн нөлөө
8	TLB-ийн хэмжээ (locality)	256 fr, TLB 1–64	TLB hit rate-ийн ханалт
9	GUI: анхдагч 12 хаяг, reference string	128 / 3 fr	Fault, TLB hit, солилтын дүрслэл

7. Туршилтын арга

Зүйл	Тодорхойлолт
Орчин	Ubuntu 24.04, Python 3.12 / 3.13, Tk 8.6. GUI-г Xvfb (virtual display) дээр бодитоор ажиллуулж screenshot авсан
Номын өгөгдөл	`examples/book/`: `addresses.txt` (1000 хаяг), `BACKING_STORE.bin` (65 536 байт), `correct.txt` (хүлээгдсэн гаралт). Эх сурвалж: номын зохиогчийн `github.com/greggagne/osc10e`, `ch10`
Өөрсдийн өгөгдөл	`examples/`: ref_textbook, ref_belady, boundary, bad_input, empty, addresses_1000_locality. `tools/gen_examples.py` нь seed=2026-аар дахин үүсгэнэ
Locality өгөгдөл	60–120 хандалт тутамд 12 page-ийн шинэ ажлын олонлог (working set) сонгож, хандалтын 90%-ийг түүнд, 10%-ийг санамсаргүй page-д хийнэ
Автомат тест	`python3 -m unittest discover tests -v` (24 тест)
Номын шалгалт	`python3 tools/check_correct.py [--frames 128 --policy LRU]`
Туршилтын скрипт	`python3 src/experiment.py <файл> --frames ... --csv out.csv`. Үр дүн `docs/results/*.csv`
Хүлээгдсэн үр дүн	`correct.txt`-тэй 1000/1000. Номын жишээ: FIFO 15 / LRU 12, Belady 9 → 10. Frame өсөхөд fault буурна. Locality үед LRU ≤ FIFO
Давтагдах байдал	Seed тогтмол, симуляци детерминист тул ижил оролтод үр дүн яг давтагдана

8. Үр дүн

8.1 Автомат тестийн үр дүн

`Ran 24 tests in 0.25s - OK` (`tests/test_vmm.py`).

ID	Оролт / нөхцөл	Хүлээгдсэн	Бодит	Үр дүн
T1	16916, 256 frame	page 66, offset 20, phys 20, fault	66, 20, 20, fault	PASS
T2	`0xABCD0000 | 16916`	page 66 (дээд бит хасагдана)	66	PASS
T3	16916 хоёр удаа	2 дахь нь TLB hit	hit	PASS
T4	Page 0 1 0 2 1, 2 frame	FIFO 3, LRU 4 fault	3, 4	PASS
T5	Page 0 1 0 1, 1 frame, LRU	4 fault	4	PASS
T6	Page 5 × 10, 4 frame	1 fault	1	PASS
T7	1 frame: 0, 256, 0	Сүүлийнх нь TLB miss (evict)	miss	PASS
T8	Store-д 200 байт	Утга −56	−56	PASS
T9	`1 abc 99999999999`; frame 0/300, policy X, TLB 0, store 1 байт	2 оролтын алдаа, 5 `VMMError`	2, 5	PASS
T10	CLI: 3 хаяг, байхгүй файл	Page Faults = 2; exit 2	2; 2	PASS
T11	Номын ref string, 3 frame	FIFO 15, LRU 12	15, 12	PASS
T12	Belady ref string	FIFO 9 → 10, LRU буурна	9 → 10, 10 → 8	PASS
T13	0, 65535, 65536, $2^{32}-1$	(0,0), (255,255), page 0, page 255	таарсан	PASS

ID	Оролт / нөхцөл	Хүлээгдсэн	Бодит	Үр дүн
T14	refs: `0 5 255, 256`	0, 1280, 65280; 256 → алдаа	таарсан	PASS
T15	`12 ²` ба араб цифр	[12], 2 алдаа	[12], 2	PASS
T16	4 frame, page 0–3, дараа нь 4	Эхний 4-т солилтгүй, дараа нь P0 гарна	таарсан	PASS
T17	676 хаяг, 16 frame, LRU	$0 \le$ phys $<$ 4096, phys = fr·256+off	бүгд	PASS
T18	Тусгай store, 8 frame	Утга = store[addr]	таарсан	PASS
T19	CLI: `1 abc -5`, хоосон, binary файл	exit 2 (унахгүй)	2, 2, 2	PASS
T20	CLI `--refs --frames 3 --policy LRU`	Page Faults = 12	12	PASS
T21	2 frame, 3 page (солилттой)	Frame-ийн 256 байт = ачаалсан page	таарсан	PASS
T22	Номын „How to begin“: 1, 256, 32768, 32769, 128, 65534, 33153	(0,1), (1,0), (128,0), (128,1), (0,128), (255,254), (129,129)	таарсан	PASS
T23	Номын өгөгдөл, 256 frame	`correct.txt`-тэй 1000 мөр ижил; 244 fault, 54 TLB hit	1000/1000; 244, 54	PASS
T24	Номын өгөгдөл, 128 frame, FIFO / LRU	Утга 1000/1000; 538 / 539 fault	1000/1000; 538, 539	PASS

8.2 Туршилтын үр дүн

Туршилт 1: сурах бичгийн correct.txt-тэй харьцуулалт

Номын 1000 хаягийг (`addresses.txt`) номын backing store-той (`BACKING_STORE.bin`) хөрвүүлж, гаралтыг `correct.txt`-тэй мөр бүрээр нь харьцуулав.

Frame	Бодлого	Таарсан	Page faults	TLB hits	Харьцуулсан
256	FIFO	1000/1000	244 (24.4%)	54 (5.4%)	Мөр бүтнээрээ (логик, физик хаяг, утга)
256	LRU	1000/1000	244 (24.4%)	55 (5.5%)	Мөр бүтнээрээ
128	FIFO	1000/1000	538 (53.8%)	54 (5.4%)	Утга (Value)
128	LRU	1000/1000	539 (53.9%)	55 (5.5%)	Утга (Value)

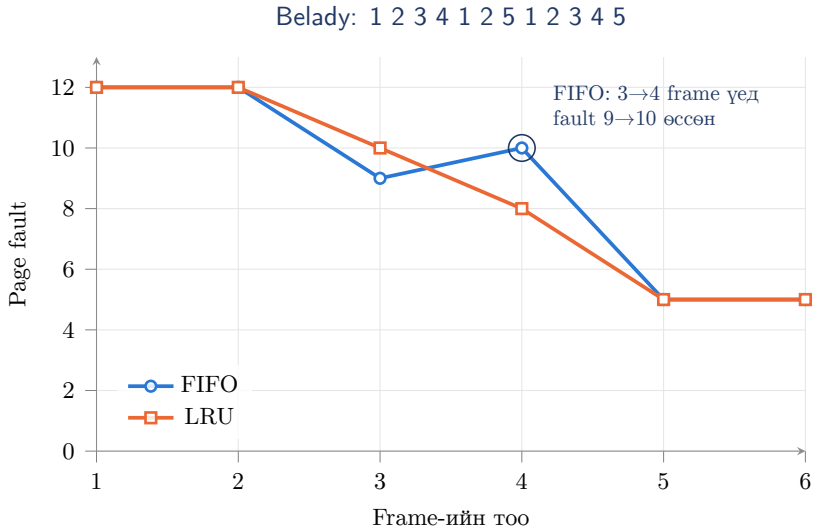
```
$ python3 tools/check_correct.py
1000/1000 мөр таарсан  [frames=256, FIFO, TLB=16]  Page Faults = 244, TLB Hits = 54
$ python3 tools/check_correct.py --frames 128 --policy LRU
1000/1000 утга (Value) таарсан  [frames=128, LRU, TLB=16]  Page Faults = 539, TLB Hits = 55
```

256 frame-тэй үед солилт гардаггүй бөгөөд хоосон frame-ийг 0-ээс дараалан олгодог тул физик хаяг ч номын хариутай яг таарсан. 128 frame-тэй үед солилтоос болж физик хаяг өөр болох тул зөвхөн утгыг харьцуулсан. Утга бүгд таарсан нь гаргасан page-ийг дахин ачаалах логик зөв ажиллаж байгааг харуулна. Шалгалт өөрөө найдвартай эсэхийг мөн туршсан. Солилтын дараа санаатайгаар өөр page хуулдаг алдаатай хувилбар 128 frame-тэй үед ердөө 753/1000 утга таарч, алдаа илэрсэн.

Туршилт 2–4: сурах бичгийн жишээнүүд

Номын „How to begin“ хэсэгт өгсөн 7 тоо (1, 256, 32768, 32769, 128, 65534, 33153) бүгд зөв задарсан (T22). Reference string-ийн жишээнүүдийн үр дүн:

Туршилт	FIFO	LRU	Номын хариу	Дүгнэлт
Номын ref string, 3 frame	15	12	15 / 12	Таарсан
Belady, 3 frame	9	10	FIFO 9	Belady-ийн аномали давтагдсан
Belady, 4 frame	10	8	FIFO 10	LRU 10 → 8 (аномалигүй)



Зураг 5. Belady-ийн аномали: FIFO-д 3 → 4 frame болоход fault 9 → 10 болж өссөн

Туршилт 5–6: захын утга ба алдаатай оролт

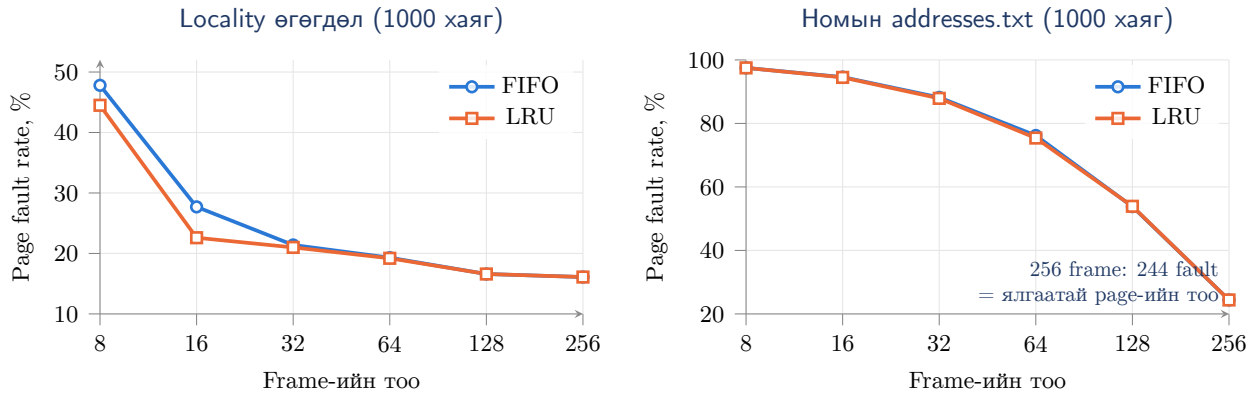
```
$ python3 src/cli.py examples/boundary.txt --frames 128 --store examples/book/BACKING_STORE.bin
Virtual address: 0 Physical address: 0 Value: 0
Virtual address: 255 Physical address: 255 Value: 63
Virtual address: 256 Physical address: 256 Value: 0
Virtual address: 65535 Physical address: 767 Value: -1
Virtual address: 65536 Physical address: 0 Value: 0
Virtual address: 65791 Physical address: 255 Value: 63
Virtual address: 4294967295 Physical address: 767 Value: -1
Number of Translated Addresses = 7
Page Faults = 3   ...   TLB Hits = 4

$ python3 src/cli.py examples/bad_input.txt
Алдаа: Хаягийн алдаа: #2 "abc" сөрөг биш бүхэл тоо биш; #3 "-5" сөрөг биш бүхэл тоо биш;
  #4 "3.14" ...; #5 "99999999999" 32 битийн хязгаар 0..4294967295-аас хэтэрсэн;
  #6 "0x1F" сөрөг биш бүхэл тоо биш (+1 бусад)                              [exit 2]
```

65535 нь page 255, offset 255 болсон. 65536 болон 65791-ийн дээд бит хасагдаж page 0 руу буусан тул TLB hit өгсөн. $2^{32} - 1$ ч мөн page 255 болсон. Алдаатай оролтын үед програм унахгүй, алдаа бүрийг дугаар ба шалтгааны хамт мэдээлсэн.

Туршилт 7: frame-ийн тооны нөлөө (1000 хаяг, TLB = 16)

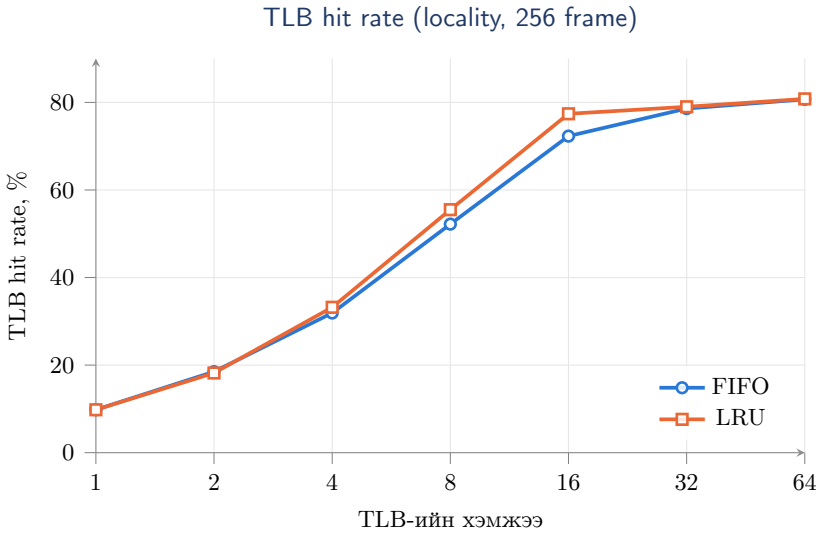
Frame	Locality FIFO	Locality LRU	Номын хаяг FIFO	Номын хаяг LRU
8	47.8%	44.5%	97.5%	97.5%
16	27.7%	**22.6%**	94.6%	94.5%
32	21.4%	21.0%	88.2%	87.9%
64	19.3%	19.2%	76.2%	75.4%
128	16.6%	16.6%	53.8%	53.9%
256	16.1%	16.1%	24.4%	24.4%



Зураг 6. Page fault rate: зүүн талд locality-тэй өгөгдөл, баруун талд номын `addresses.txt`

Туршилт 8: TLB-ийн хэмжээ (locality, 256 frame)

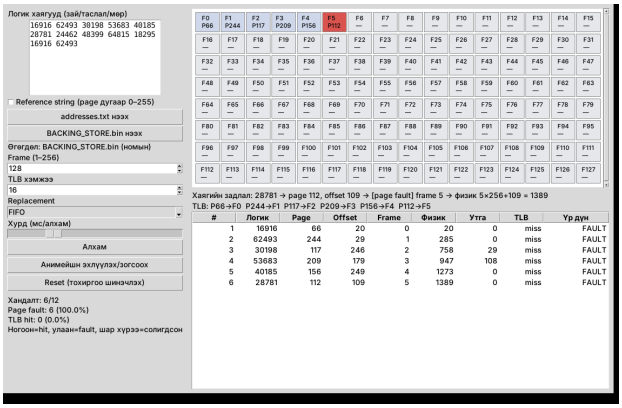
TLB	1	2	4	8	16	32	64
FIFO	9.8%	18.5%	31.9%	52.2%	72.3%	78.6%	80.7%
LRU	9.8%	18.2%	33.2%	55.5%	77.4%	79.0%	80.8%



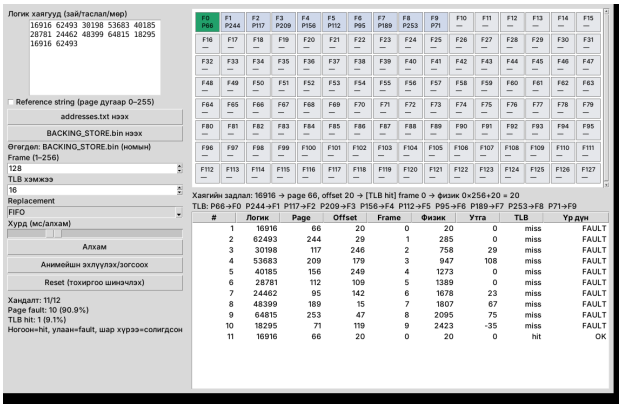
Зураг 7. TLB hit rate 16 оролт хүртэл огцом өсөөд, 32-оос хойш ойролцоогоор 80%-д ханасан

8.3 Screenshot (Xvfb дээр бодитоор ажиллуулж авсан)

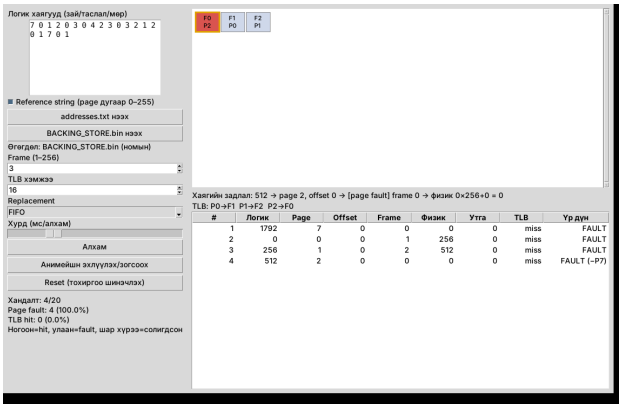
Зураг 4-ээс гадна дараах төлөвүүдийг бодитоор ажиллуулж авсан (Туршилт 9). GUI нь номын `BACKING_STORE.bin`-ийг ачаалсан байна:



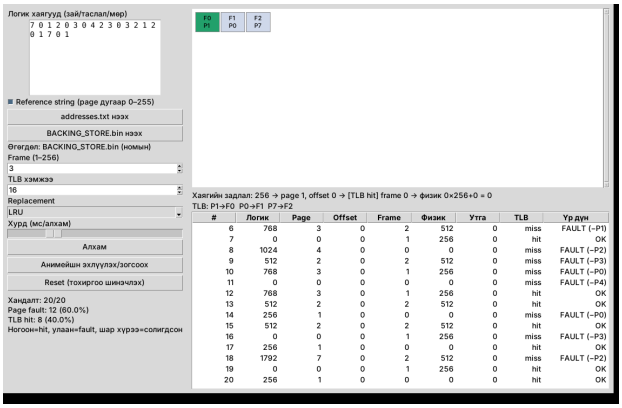
(a) Анхдагч 12 хаяг, 6 дахь алхам: page fault (улаан)



(b) 11 дэх алхам: 16916 дахин ирж TLB hit (ногоон). Утгууд (0, 0, 29, 108, ...) `correct.txt`-тэй таарна

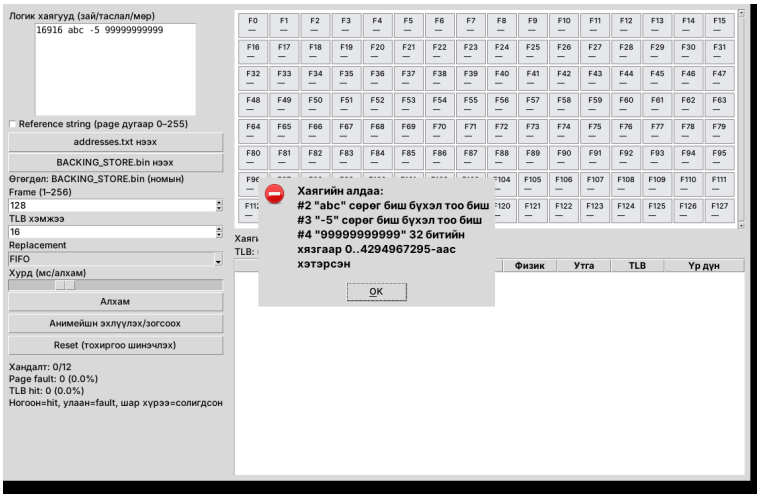


(c) FIFO, 3 frame: P7-г гаргасан (шар хүрээ, `FAULT (-P7)`)



(d) LRU, 3 frame: 20 хандалтаас 12 fault, номтой таарсан

Зураг 8. GUI-ийн бодит төлөвүүд



Зураг 9. Алдаатай оролтыг дугаар болон шалтгааны хамт мэдээлж байна

9. Хэлэлцүүлэг

9.1 Үр дүнгийн тайлбар

- **Зөв ажиллагаа.** Номын `correct.txt`-тэй 1000/1000 мөр таарсан. Энэ нь хаяг задлах, TLB, page table, demand paging, backing store-оос унших, signed byte хөрвүүлэлт бүгд зөв ажиллаж байгааг харуулна. 128 frame-тэй үед ч утга бүгд зөв гарсан нь солилт болон дахин ачаалах логик зөв гэдгийг нотолно. Номын FIFO/LRU жишээ ч тоогоороо таарсан.
- **LRU ба FIFO (locality).** Locality-тэй ачаалалд, ажлын олонлог frame-д арай ядан багтах үед LRU илт давуу байв (16 frame-тэй үед 22.6% vs 27.7%). Учир нь FIFO байнга хэрэглэгдэж буй „хуучин“ page-ийг гаргадаг. Frame хангалттай олон болоход хоёр бодлого хоёулаа зөвхөн анхны (cold) fault-тай болж ижилсдэг. 256 frame-тэй үед гарсан 161 fault нь өгөгдөл дэх ялгаатай page-ийн тоотой тэнцүү.
- **Номын хаяг (санамсаргүй).** Номын хаягууд санамсаргүй үүсгэгдсэн тул locality байхгүй. Ийм үед ирээдүйн хандалтыг өнгөрснөөр таах боломжгүй тул LRU-гийн давуу тал алга болно. 128 frame-тэй үед LRU (539) нь FIFO-оос (538) ч нэгээр илүү fault гаргасан. Fault rate ойролцоогоор $1 - \text{frames}/256$ байв (64 frame $\rightarrow$ 76.2%). 256 frame-тэй үеийн 244 fault нь `addresses.txt` дахь ялгаатай page-ийн тоотой яг тэнцүү.
- **TLB.** Locality-тэй өгөгдөлд 16 оролттой TLB хандалтын 3/4-ийг page table руу хандалгүй хөрвүүлсэн. Ажлын олонлог 12 page тул 32-оос том TLB бараг ашиг өгсөнгүй. Харин номын санамсаргүй хаягуудад TLB hit ердөө 5.4% байв. Ном өөрөө ч эдгээр хаягуудад locality байхгүй тул TLB hit rate өндөр гарахгүй гэж тэмдэглэсэн байдаг.
- **Belady.** FIFO нь stack алгоритм биш тул frame нэмэхэд fault өсөж болохыг харууллаа. LRU-д ийм үзэгдэл ажиглагдаагүй.

9.2 Хязгаарлалт ба мэдэгдэж буй асуудлууд

- **Хамрах хүрээ.** Бичих үйлдэл (dirty bit), олон процесс, хоёр түвшний page table хэрэгжээгүй.
- **TLB хайлт.** TLB-д шугаман хайлт хийдэг ($O(T)$). Бодит TLB зэрэгцээ (associative) хайдаг тул энэ нь симуляцийн хялбарчлал юм. TLB-ийн бодлогыг page replacement-ээс тусад нь сонгох боломжгүй.
- **Locality өгөгдөл.** Өөрсдийн загварчилсан (synthetic) өгөгдөл тул бодит програмын хандалтыг бүрэн төлөөлөхгүй.
- **GUI.** tkinter шаардлагатай (Linux дээр `python3-tk`). Тохиргоо өөрчилсний дараа Reset дарах хэрэгтэй.

10. Дүгнэлт

Сурах бичгийн Programming Project-ийн үндсэн функц бүрэн хэрэгжсэн: хаяг задлах, 16 оролттой TLB, page table, физик санах ой ба demand paging, 1–256 frame-тэй үеийн FIFO ба LRU солилт, номын форматтай гаралт. Програмын зөв ажиллагааг номын албан ёсны `correct.txt`-ээр баталгаажуулсан: 1000/1000 мөр таарсан. Багшийн шаардсан **page/frame animation** цөмтэй шууд холбогдсон. Хэрэглэгч хаяг/reference string, frame, TLB, policy-г сольж, алхам эсвэл анимейшнээр ажиглах, reset хийх боломжтой.

Номын жишээтэй таарсан үр дүн (FIFO 15 / LRU 12, Belady 9 $\rightarrow$ 10) гарсан. Мөн locality-тэй ачаалалд LRU давуу байдаг, санамсаргүй хаягт давуу тал нь алга болдог, TLB 16–32 оролтод ханадаг гэдгийг тоон үр дүнгээр нотоллоо. Буруу болон захын оролтыг ойлгомжтой мэдээлж, 24 автомат тест бүгд амжилттай давсан. Цаашид dirty bit ба write-back, тусдаа TLB бодлого

нэмэх, бодит програмын хандалтын trace-ээр турших боломжтой.

Ашигласан материал

- A. Silberschatz, P. B. Galvin, G. Gagne. *Operating System Concepts*, 10th ed. Wiley, 2018. Chapter 10, Programming Project „Designing a Virtual Memory Manager“.
- G. Gagne. *OSC10e source code*, ch10 (`addresses.txt`, `BACKING_STORE.bin`, `correct.txt`). `github.com/greggagne/osc10e`
- L. A. Belady, R. A. Nelson, G. S. Shedler. „An anomaly in space-time characteristics of certain programs running in a paging machine.“ *Communications of the ACM*, 12(6), 1969.
- Python Software Foundation. *Python 3 documentation*: tkinter, argparse, unittest. `docs.python.org/3`
- F.CSM302 „Бие даалт 1“ заавар ба „Сэдэв ба шүүмжийн хуваарилалт“, 2026.
- Төслийн код: `github.com/ws202-gg/Designing-a-Virtual-Memory-Manager`

Хавсралт А. Ажлын төлөвлөгөө

Талбар	Мэдээлэл
Хуваарилалтын код	II-4-15
Төсөл	Designing a Virtual Memory Manager (page/frame animation)
Repository	`github.com/ws202-gg/Designing-a-Virtual-Memory-Manager`
Хэл/сан/орчин	Python 3.10+, tkinter, unittest. Linux / Windows / macOS
Үндсэн дүрслэл	Page/frame animation (frame grid, TLB, hit / fault / evict өнгө)

Гишүүн	Хариуцах ажил	Commit / file
Э.Мөнх-Оргил /B242270130/	Багийн ахлагч. VMM цөм (хаяг задлах, TLB, page table, FIFO/LRU)	`src/vmm.py`
Д.Ундармаа /B242270044/	Interactive UI (page/frame animation)	`src/gui.py`
В.Хүсэлбилэг /B241910007/	Консол програм, тайлан (Report)	`src/cli.py`, `docs/…Report.pdf`
С.Сүхбат /B241940044/	Автомат тест, boundary/error тест, GitHub repository ба Release v1.0	`tests/`, `tools/check_correct.py`, `README.md`
М.Хатансайхан /B231940049/	Туршилтын скрипт, туршилтын өгөгдөл, screenshot, график	`src/experiment.py`, `examples/`, `docs/img/`

Тест	Оролт/нөхцөл	Хүлээгдсэн үр дүн
1	Номын өгөгдөл (`examples/book/`), 256 frame	`correct.txt`-тэй 1000/1000 мөр. 244 fault, 54 TLB hit
2	Номын өгөгдөл, 128 frame, FIFO / LRU	Утга 1000/1000. 538 / 539 fault
3	ref_textbook, 3 frame, FIFO / LRU	15 / 12 fault
4	ref_belady, FIFO, 3 ба 4 frame	9 ба 10 fault
5	boundary.txt	65536 → page 0. Програм унахгүй
6	bad_input.txt, empty.txt	Алдааны мессеж, exit 2
7	addresses_1000_locality, 16 frame	LRU 22.6% < FIFO 27.7%